



Trabalho 1: Pilha Dinâmica

Disciplina: Estruturas de Dados

Professor: Wagner de Almeida Junior

Valor: 2 pontos

A partir da implementação com vetor estático vista em aula, seu grupo deverá criar uma nova versão da Pilha que **não possua um limite de tamanho fixo**. Para isso, o vetor que armazena os itens da pilha deverá ser alocado dinamicamente no **Heap**, e sua capacidade deverá aumentar automaticamente sempre que a pilha estiver cheia.

O projeto deve ser modularizado em, no mínimo, três arquivos: **pilha_dinamica.h**, **pilha_dinamica.c** e **main.c**. O cabeçalho (**pilha_dinamica.h**) deve declarar a seguinte interface pública:

```
#pragma once

typedef struct {
    int *itens;
    int topo;
    int capacidade;
} PilhaDinamica;

// Cria uma nova pilha na Heap com uma capacidade inicial.
PilhaDinamica* criar_pilha(int capacidade_inicial);

// Libera toda a memória associada à pilha.
void destruir_pilha(PilhaDinamica *p);

// Adiciona um elemento no topo da pilha.
void push(PilhaDinamica *p, int valor);

// Remove e retorna o elemento do topo da pilha.
int pop(PilhaDinamica *p);

// Apenas retorna o elemento do topo, sem remover.
int peek(PilhaDinamica *p);

// Verifica se a pilha está vazia.
int esta_vazia(PilhaDinamica *p);
```





A função push é fundamental neste trabalho. Ela deve verificar se a pilha está cheia antes de inserir um novo elemento.

- Se estiver cheia, a função deve **dobrar a capacidade** do vetor itens usando realloc.
- **Importante:** Implemente o padrão de segurança do realloc, usando um ponteiro temporário para não perder a referência do vetor original em caso de falha na realocação.

Depois de construir e testar a pilha, vocês a usarão para resolver um problema.

- **O Problema:** Converter um número inteiro positivo da base 10 (decimal) para a base 2 (binário).
- **Por que usar uma Pilha?** O algoritmo de conversão por divisões sucessivas gera os bits do número binário na **ordem inversa** (do menos significativo para o mais significativo). A propriedade **LIFO (Last-In, First-Out)** da pilha é perfeita para armazenar esses bits e depois recuperá-los na ordem correta.
- **O Algoritmo que você deve implementar em main.c:**
 1. Peça ao usuário um número inteiro positivo n .
 2. Crie uma instância da sua Pilha Dinâmica.
 3. Enquanto n for maior que 0:
 - Calcule o resto da divisão de n por 2.
 - Empilhe (push) o resto na sua pilha.
 - Atualize n com o resultado da divisão inteira por 2 ($n = n / 2$).
 4. Após o laço, enquanto a pilha não estiver vazia, desempilhe cada bit e imprima-o na tela.

Entregáveis

1. **Código-Fonte Completo:** Os arquivos pilha_dinamica.h, pilha_dinamica.c e um main.c que demonstre todas as funcionalidades, incluindo o redimensionamento automático.
2. **Apresentação Oral (aproximadamente 10 minutos):**
 - Uma breve demonstração do programa funcionando.
 - Uma explicação do código das funções principais, incluindo o detalhamento da lógica de verificação de capacidade e o uso do realloc na função push.